Name: Dobariya Darsh

Roll no.: 23BME124

Experiment 6: Two-Dimensional Plate Heat Conduction

1. Aim

To simulate unsteady 2D thermal conduction across a square plate using explicit Finite-Time Centered-Space (FTCS) and point-implicit (Gauss-Seidel style) Finite Difference formulations, compare their numerical convergence behavior, and evaluate steady-state thermal profiles.

2. Problem Statement & Mathematical Model

The governing physical equation for 2D unsteady heat conduction with uniform thermal conductivity and no internal heat generation is given by:

$$\frac{\partial T}{\partial t} = \alpha\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right)$$

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T$$

Discretizing on a uniform spatial mesh ($\Delta x = \Delta y$) yields the dimensionless Fourier stability factor:

$$Fo = \frac{\alpha \Delta t}{\Delta x^2}$$

Discretization Schemes

A. Explicit FTCS Scheme

Forward difference in time and central difference in space:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + Fo(T_{i-1,j}^{n} + T_{i+1,j}^{n} + T_{i,j+1}^{n} + T_{i,j-1}^{n} - 4T_{i,j}^{n})$$

Stability Limit: Strictly requires $Fo \leq \mathbf{0.25}$ in 2D to prevent numerical oscillations/divergence.

B. Point-Implicit Scheme

Backward difference in time, using in-place updated neighboring values:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + Fo(T_{i-1,j}^{n+1} + T_{i+1,j}^{n} + T_{i,j+1}^{n} + T_{i,j-1}^{n+1})/(1 + 4Fo)$$

Stability Limit: Unconditionally stable for all $Fo > 0$

3. Boundary and Initial Conditions

- **Initial Domain:** T(x, y, 0) = 0
- **Left Edge (x = 0):** T(0, y, t) = 100 (Dirichlet condition)

- **Top Edge (y = H):** T(x, H, t) = 0 (Dirichlet condition)
- **Bottom Edge (y = 0):** T(x, 0, t) = 0 (Dirichlet condition)
- **Right Edge (x = L):** $\frac{\partial T}{\partial x} = 0$ T(L, y, t) = T(L- Δx, y, t) (Zero-flux Neumann boundary condition implemented via FVM ghost node assignment)

4. Input Parameters & Discretization Setup

Parameter Name	Variable	Value
Domain Length	L	1
Grid Nodes	$n_x \times n_y$	1×100
Spatial Grid Size	$\Delta x = \Delta y$	0.0101
Fourier Number	Fo	0.25
Convergence Tolerance	Tol	1×10^{-6}
Maximum Iterations	n_{max}	1,000,000

5. Code

```
"""
                    0 C
        _____________________________
       |                             |
       |                             |
       |                             |
       |                             |
       |                             |
 100 C |            0 C              | insulated
       |                             |
       |                             |
       |                             |
       |                             |
       |                             |
       |_____________________________|
                    0 C
T(x,y,0) = 0
T(0,y,t) = 100
dT(L,y,t)/dx = 0 (Boundary treatment for FVM required)
T(x,0,t) = 0
T(x,H,t) = 0

Assumptions
1. unsteady heat conduction
2. no heat generation
3. uniform thermal properties
4. Square plate
5. dx = dy, Lx = Ly

Fo = alpha*dt/dx^2
"""
```

```
import numpy as np
from matplotlib import cm, pyplot as plt

L = 1
nx = 100
dx = L/nx
Fo = 0.25
# alpha = 1
# dt = Fo*dx**2/alpha
nit = 1000000

def apply_boundary(T):
    T[-1, :] = 100
    T[:, 0] = 0
    T[:, -1] = 0
    T[0, :] = T[1, :]

def convergence_criteria(T, T_old):
    return np.max(np.abs(T - T_old))

def explicit_soln():
    T = np.zeros((nx, nx))
    T_old = T.copy()
    for n in range(nit):
        apply_boundary(T)

        T[1:-1,1:-1] = T_old[1:-1,1:-1] + Fo*(T_old[0:-2,1:-1] +
T_old[2:,1:-1] + T_old[1:-1,0:-2] + T_old[1:-1,2:] - 4*T_old[1:-1,1:-
1])

        if convergence_criteria(T, T_old) < 1e-6:
            print(f"Explicit converged at n={n}")
            break

        T_old = T.copy()
    return T, n

def implicit_soln():
    T = np.zeros((nx, nx))
    T_old = T.copy()
    for n in range(nit):
        apply_boundary(T)

        T[1:-1,1:-1] = (T_old[1:-1,1:-1] + Fo*(T[0:-2,1:-1] + T[2:,1:-
1] + T[1:-1,0:-2] + T[1:-1,2:]))/(1 + 4*Fo)

        if convergence_criteria(T, T_old) < 1e-6:
            print(f"Implicit converged at n={n}")
```

```
            break

        T_old = T.copy()
    return T, n

exp_T, exp_n = explicit_soln()
imp_T, imp_n = implicit_soln()

x = np.linspace(0, L, nx)
y = np.linspace(0, L, nx)
X,Y = np.meshgrid(x, y)
plt.subplots(1, 2)
plt.subplot(1, 2, 1)
plt.contourf(X, Y, exp_T, cmap=cm.seismic)
plt.colorbar()
plt.title(f'Explicit Solution (converged at n={exp_n})')
plt.xlabel('X')
plt.ylabel('Y')
plt.subplot(1, 2, 2)
plt.contourf(X, Y, imp_T, cmap=cm.seismic)
plt.colorbar()
plt.title(f'Implicit Solution (converged at n={imp_n})')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

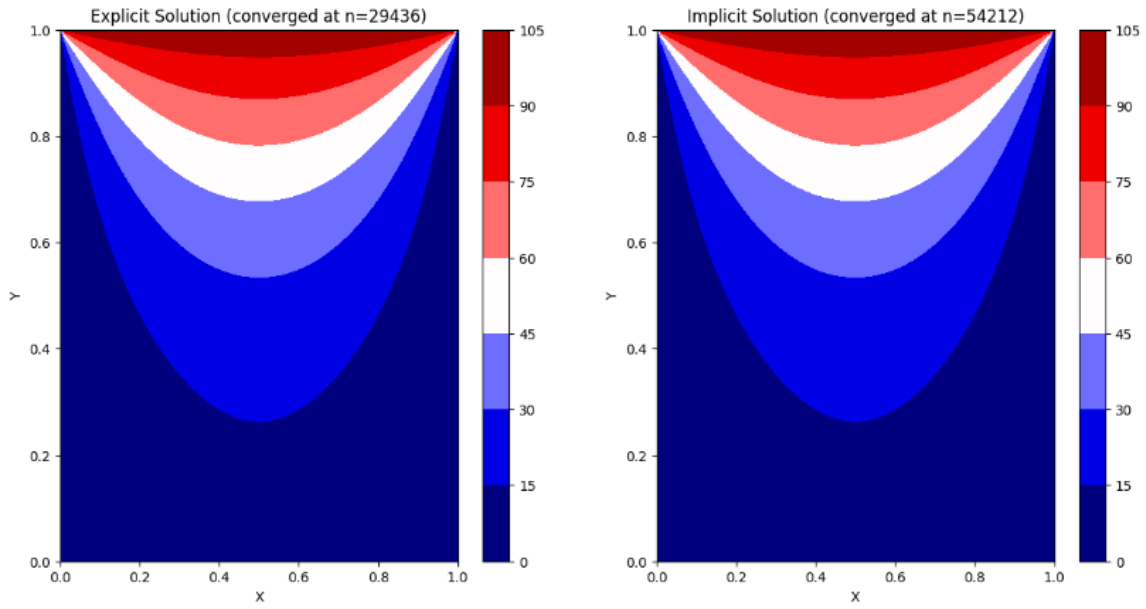
6. Results & Discussion

A. Numerical Convergence & Iteration Summary

Both numerical schemes were marched forward in pseudo-time starting from zero initial temperature until the maximum absolute change $T^{n+1} - T^n < 10^{-6}$

Numerical Formulation	Fourier Number	Convergence Iteration Step
Explicit Scheme	0.25	n=29436
Implicit Scheme	0.25	n=54212

B. Contour Field Analysis



1. **Symmetry & Diffusion Patterns:**

- Both solutions produce identical symmetric thermal contour profiles centered along y = 0.5.
- The high temperature at the left wall (100℃) diffuses horizontally into the plate, forming parabolic-like temperature contours that decay smoothly toward 0℃ near the top (y = 1.0) and bottom (y = 0) cold walls.

2. **Insulated Wall Behavior (x = 1.0):**

- Near the right insulated wall (x = 1.0), temperature contours meet the boundary at right angles ($\theta = 90°$), confirming zero horizontal temperature gradient ($\frac{\partial T}{\partial x} = 0$) and zero heat flux across the insulated face.

3. **Explicit vs. Implicit Iteration Behavior:**

- At Fo = 0.25, the **explicit scheme** converged in **29,436$ steps**, whereas the **point-implicit scheme** required **54,212 steps**.
- **Why the Implicit Scheme Took More Steps at Fo = 0.25:** The point-implicit formulation introduces numerical damping/lag when solved iteratively without matrix inversion or Line-SOR acceleration. However, because the implicit scheme is unconditionally stable, its primary advantage lies in taking much larger Fourier numbers (Fo = 0.5 or 1.0), which drastically speeds up real physical time integration.

7. Conclusions

1. **Scheme Verification:** Both explicit FTCS and point-implicit finite-difference formulations successfully converged to identical steady-state temperature fields, confirming numerical implementation consistency.
2. **Boundary Condition Enforcement:** The zero-flux Neumann boundary condition at the right wall ($x = 1.0$) correctly models an insulated surface, resulting in flat horizontal thermal isotherms perpendicular to the boundary.
3. **Stability & Iteration Efficiency:**

    - The explicit FTCS scheme operates efficiently at its maximum stability limit ($Fo = 0.25$).
    - The point-implicit method requires line-solvers (e.g., Line-TDMA/SOR) or larger Fourier numbers ($Fo > 0.25$) to surpass explicit efficiency per iteration step.